UniSENAI

Trilha de Banco de Dados

Módulo 06: Índices e Otimização de Consultas

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.

2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.

3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.

4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.

5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.

6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.

7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.

8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 6: Índices e Otimização de Consultas

Os **índices** são um dos principais mecanismos para melhorar a **performance de consultas SQL**, especialmente em bancos de dados com grande volume de dados. Este módulo aborda a criação de índices, a análise do plano de execução das consultas e a aplicação de **boas práticas de otimização**, fundamentais para sistemas em produção.

---

1. Índices: Conceito e Importância

O que são Índices

Um **índice** é uma estrutura de dados criada pelo banco de dados para **acelerar operações de busca**, funcionando de maneira semelhante ao índice de um livro: em vez de percorrer todas as páginas, o sistema vai direto à informação desejada.

Por que usar índices?

- Aceleram consultas `SELECT`
- Melhoram desempenho em cláusulas `WHERE`, `JOIN` e `ORDER BY`
- Reduzem o número de linhas analisadas em tabelas grandes
- Diminuem o tempo de resposta da aplicação

Atenção ao Uso Excessivo

Apesar dos benefícios, índices possuem custo:

- Aumentam o tempo de `INSERT`, `UPDATE` e `DELETE`
- Consomem espaço em disco
- Exigem manutenção automática pelo banco

⚠️ **Regra prática**: indexe colunas muito consultadas, não colunas muito alteradas.

---

2. Criação de Índices

2.1 CREATE INDEX - Índice Simples

Cria um índice em uma ou mais colunas.

Exemplo Prático: Índice em uma coluna `nome` na tabela `clientes`:

```
CREATE INDEX idx_nome_cliente ON clientes(nome);
```

- **`idx_nome_cliente`: Nome do índice.**

- `clientes(nome)`: **Define que o índice será criado na coluna `nome`.**

Onde usar

- Filtros frequentes por nome
- Ordenações por nome

2.2 UNIQUE INDEX - Índice Único

Garante que não existam valores duplicados.

Exemplo Prático: Índice único na coluna `email`:

```
CREATE UNIQUE INDEX idx_email_cliente ON clientes(email);
```

Aplicabilidade: Garantir unicidade em colunas críticas, como emails ou números de identificação.

2.3 Índices Compostos

Índices compostos envolvem **duas ou mais colunas** e são extremamente eficientes quando a consulta utiliza **a mesma ordem das colunas do índice**.

Exemplo Prático: Criar um índice composto em `nome` e `cidade` na tabela `clientes`:

```
CREATE INDEX idx_nome_cidade ON clientes(nome, cidade);
```

Importante

- Este índice é usado para:
  - `WHERE nome = ?`
  - `WHERE nome = ? AND cidade = ?`
- Não é usado para:
  - `WHERE cidade = ?` (ordem incorreta)

Aplicabilidade:

- Consultas que utilizam ambas as colunas no filtro ou ordenação.
- Exemplo de consulta otimizada:

```
SELECT *
FROM clientes
WHERE nome = 'João Silva' AND cidade = 'São Paulo';
```

---

3. Análise de Performance com EXPLAIN

O comando `EXPLAIN` exibe o plano de execução de uma consulta, indicando como o banco de dados planeja executar a operação.

Exemplo Prático: Verificar a performance de uma consulta com `EXPLAIN`:

```
EXPLAIN SELECT * FROM clientes WHERE nome = 'João Silva';
```

Saída Interpretada:

- **`type`**: Tipo de busca (ideal: `index` ou `const`).
- **`key`**: Índice usado na consulta.
- **`rows`**: Número de linhas examinadas (quanto menor, melhor).
- **`Extra`**: Informações adicionais (evite `Using full table scan`).

---

4. Índices em JOINs e WHERE

Índices são especialmente úteis em consultas complexas que utilizam `JOIN` ou `WHERE`.

Exemplo Prático:

Imagine duas tabelas, `clientes` e `pedidos`, relacionadas por `id_cliente`. Um índice na coluna `id_cliente` pode acelerar a consulta.

Sem Indice:

```
SELECT *
FROM clientes
JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente;
```

Com Índice:

```
CREATE INDEX idx_id_cliente ON pedidos(id_cliente);
```

Resultado:

- O plano de execução mostrará uma redução no número de linhas analisadas.

---

5. Otimização de Consultas SQL

Os índices são usados automaticamente pelo banco de dados em consultas que envolvem colunas indexadas. Isso reduz o número de registros analisados.

Exemplo Prático:

Consulta com índice na coluna `nome`:

```
SELECT * FROM clientes WHERE nome = 'Maria Souza';
```

- **Sem índice:** O banco faz uma **varredura completa da tabela** (*Full Table Scan*).
- **Com índice:** Apenas os registros correspondentes são acessados diretamente.

5.1 Evitando Full Table Scan

Um **Full Table Scan** ocorre quando o banco precisa analisar todas as linhas da tabela.

Estratégias para evitar:

- Criar índices adequados
- Usar filtros em colunas indexadas
- Evitar funções sobre colunas indexadas

Estratégias para Evitar:

1. **Criação de Índices Apropriados**:

```
CREATE INDEX idx_cidade ON clientes(cidade);
```

2. **Uso Correto de Condições**: Sempre utilize colunas indexadas em filtros `WHERE`:

```
SELECT * FROM clientes WHERE cidade = 'São Paulo';
```

3. **Evitar operações em Colunas**: Operações nas colunas podem impedir o uso do índice.

- **Ineficiente**:

```
SELECT * FROM clientes WHERE UPPER(nome) = 'JOÃO SILVA';
```

- **Eficiente:**

```
SELECT * FROM clientes WHERE nome = 'João Silva';
```

5.2 Limitar Registros Retornados

Use `LIMIT` para reduzir o número de registros processados:

```
SELECT * FROM clientes LIMIT 10;
```

---

6. Boas Práticas de Escrita de Consultas

- **Selecione Apenas as Colunas Necessárias**:
  - **Ineficiente**

```
SELECT * FROM clientes;
```

  - **Eficiente**

```
SELECT nome, cidade FROM clientes;
```

6.1 Evite Subconsultas Desnecessárias:

1. Substitua subconsultas por **JOINs** sempre que possível.
   a. **Ineficiente**:

```
SELECT nome FROM clientes WHERE id_cliente = (SELECT id_cliente FROM pedidos WHERE valor_total > 100);
```

   b. **Eficiente:**

```sql
SELECT clientes.nome
FROM clientes
JOIN pedidos ON clientes.id_cliente = pedidos.id_cliente
WHERE pedidos.valor_total > 100;
```

6.2 Utilize Funções de Agregação Com Cuidado:

- Agrupe e filtre somente onde necessário:

```sql
SELECT cidade, COUNT(*) AS total_clientes
FROM clientes
GROUP BY cidade
HAVING total_clientes > 10;
```

4. **Análise Regularmente a Performance**: Use `EXPLAIN` para identificar gargalos e ajustar índices e consultas.

Aplicabilidade Prática

- **Sistemas de Vendas**:
  - Criar índices em colunas como `id_cliente` e `data_pedido` para acelerar relatórios.
- **E-commerce**:
  - Melhorar a performance de consultas em catálogos com índices em `categoria` e `preco`.
- **Gestão de Projetos**:
  - Índices compostos em colunas como `id_projeto` e `id_funcionario` para relatórios de produtividade.

Com o uso correto de índices e práticas otimizadas, é possível obter consultas rápidas e eficientes, garantindo o desempenho de aplicações mesmo em grandes volumes de dados.

Questões Práticas (10)

1. **Criação de Índices**:
   Crie um índice básico na coluna `nome` da tabela `clientes` e explique como esse índice pode melhorar a performance em uma consulta `SELECT`.
2. **Índice Único**:
   Crie um índice único na coluna `email` da tabela `clientes` para garantir que não

existam emails duplicados. Explique a importância desse índice para a integridade dos dados.

3. **Índices Compostos**:
   Crie um índice composto nas colunas `nome` e `cidade` da tabela `clientes`. Em seguida, explique como esse índice pode otimizar uma consulta que filtra clientes por nome e cidade simultaneamente.
4. **Análise de Performance com EXPLAIN**:
   Execute o comando `EXPLAIN` em uma consulta que utiliza um `JOIN` entre as tabelas `clientes` e `pedidos`. Interprete os seguintes campos da saída: `type`, `key`, `rows` e `Extra`.
5. **Uso de Índices em JOINs**:
   Crie um índice na coluna `id_cliente` da tabela `pedidos` e demonstre como ele melhora a performance de uma consulta que realiza um `JOIN` entre `clientes` e `pedidos`.
6. **Evitar Full Table Scans**:
   Crie um índice na coluna `nome` da tabela `clientes` e compare o plano de execução de uma consulta `SELECT` com e sem índice.
7. **Limitação de Registros com LIMIT**:
   Execute uma consulta que retorne apenas os 5 primeiros clientes da tabela `clientes`, ordenados por `nome`. Explique como o uso de `LIMIT` pode melhorar a performance.
8. **Evitar Operações em Colunas**:
   Otimize uma consulta que filtra clientes usando `UPPER(nome)` para evitar operações que impeçam o uso de índices. Explique a mudança feita.
9. **Criação de Índices Apropriados**:
   Crie índices nas colunas `categoria` e `preco` da tabela `produtos`. Explique como esses índices podem melhorar consultas que envolvem filtros nessas colunas.
10. **Identificação de Gargalos com EXPLAIN**:
    Analise uma consulta complexa utilizando `EXPLAIN`, identifique possíveis gargalos no desempenho e proponha melhorias usando índices ou reestruturação da consulta.

Questões Teóricas (10)

1. **Conceito de Índices**:
   Explique o que são índices em SQL e por que eles são importantes para a performance de consultas em tabelas grandes.
2. **Tipos de Índices**:
   Diferencie entre um índice básico (`CREATE INDEX`) e um índice único (`CREATE UNIQUE INDEX`). Dê exemplos de aplicações práticas para cada tipo.
3. **Índices Compostos**:
   Explique o que é um índice composto e em quais situações ele é mais útil.

4. **Impacto dos Índices em Operações DML**:
   Discuta como índices podem afetar o desempenho de operações como `INSERT`, `UPDATE` e `DELETE`.
5. **EXPLAIN em Consultas SQL**:
   Qual é o objetivo do comando `EXPLAIN` no MySQL? Quais informações ele fornece para ajudar na otimização de consultas?
6. **Full Table Scan**:
   O que é um Full Table Scan e por que ele pode ser prejudicial ao desempenho? Como evitá-lo?
7. **Limitação de Registros em Consultas**:
   Por que o uso do comando `LIMIT` pode melhorar a performance de uma consulta? Cite um exemplo prático.
8. **Índices e JOINs**:
   Explique como os índices podem melhorar a performance de consultas que utilizam `JOINs`.
9. **Boas Práticas em Consultas SQL**:
   Liste e explique três boas práticas para escrever consultas SQL eficientes.
10. **Funções de Agregação e Performance**:
    Como o uso de funções de agregação (`COUNT`, `SUM`, `AVG`) pode impactar a performance de uma consulta? O que pode ser feito para otimizar consultas que utilizam essas funções?